

Problem List

1603. Design Parking System

Description

Editorial

Solutions

Submissions

Easy

Topics

Companies

Hint

Design a parking system for a parking lot. The parking lot has three kinds of parking spaces: big, medium, and small, with a fixed number of slots for each size.

Implement the `ParkingSystem` class:

- `ParkingSystem(int big, int medium, int small)` Initializes object of the `ParkingSystem` class. The number of slots for each parking space are given as part of the constructor.
- `bool addCar(int carType)` Checks whether there is a parking space of `carType` for the car that wants to get into the parking lot. `carType` can be of three kinds: big, medium, or small, which are represented by 1, 2, and 3 respectively. **A car can only park in a parking space of its `carType`.** If there is no space available, return `false`; else park the car in that size space and return `true`.

Example 1:

Input

["ParkingSystem", "addCar", "addCar", "addCar", "addCar"]
[[1, 1, 0], [1], [2], [3], [1]]

Output

[null, true, true, false, false]

Explanation

```
ParkingSystem parkingSystem = new ParkingSystem(1, 1, 0);
parkingSystem.addCar(1); // return true because there is 1 available slot for a big car
parkingSystem.addCar(2); // return true because there is 1 available slot for a medium car
parkingSystem.addCar(3); // return false because there is no available slot for a small car
parkingSystem.addCar(1); // return false because there is no available slot for a big car. It is already occupied.
```

Constraints:

- `0 <= big, medium, small <= 1000`
- `carType` is 1, 2, or 3

2.1K

71

Code

Java

Auto

```
1 class ParkingSystem {
2     int big, medium, small;
3
4     public ParkingSystem(int big, int medium, int small) {
5         this.big = big;
6         this.medium = medium;
7         this.small = small;
8     }
9
10    public boolean addCar(int carType) {
11        if (carType == 1 && big > 0) {
12            big--;
13            return true;
14        }
15        if (carType == 2 && medium > 0) {
16            medium--;
17            return true;
18        }
19        if (carType == 3 && small > 0) {
20            small--;
21            return true;
22        }
23        return false;
24    }
25 }
26
27
28 }
```

Saved

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Problem

Submissions

Leaderboard

Discussions

Editorial

The Java instanceof operator is used to test if the object or instance is an instanceof the specified type.

In this problem we have given you three classes in the editor;

- Student class
- Rockstar class
- Hacker class

In the main method, we populated an ArrayList with several instances of these classes, count method calculates how many instances of each type is present in the ArrayList. The code prints three integers, the number of instance of Student class, the number of instance of Rockstar class, the number of instance of Hacker class.

But some lines of the code are missing, and you have to fix it by modifying only 3 lines! Don't add, delete or modify any extra line.

To restore the original code in the editor, click on the top left icon in the editor and create a new buffer.

Sample Input

S
Student
Student
Rockstar
Student
Student
Hacker

Sample Output

3 1 1

```
5 class Hacker()
6
7 public class InstanceOfTutorial{
8
9     static String count(ArrayList mylist){
10         int a = 0,b = 0,c = 0;
11         for(int i = 0; i < mylist.size(); i++){
12             Object element=mylist.get(i);
13             if(element instanceof Student)
14                 a++;
15             if(element instanceof Rockstar)
16                 b++;
17             if(element instanceof Hacker)
18                 c++;
19         }
20         String ret = Integer.toString(a)+" "+ Integer.toString(b)+" "+ Integer.toString(c);
21         return ret;
22     }
23
24     public static void main(String []args){
25         ArrayList mylist = new ArrayList();
26         Scanner sc = new Scanner(System.in);
27         int t = sc.nextInt();
28         for(int i=0; i<t; i++){
29             String s=sc.next();
30             if(s.equals("Student"))mylist.add(new Student());
31             if(s.equals("Rockstar"))mylist.add(new Rockstar());
32             if(s.equals("Hacker"))mylist.add(new Hacker());
33         }
34         System.out.println(count(mylist));
35     }
36 }
```

Line: 36 Col: 2

Upload Code as File ☐ Test against custom input

Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Problem List

1396. Design Underground System

Description

Editorial

Solutions

Submissions

Medium

Topics

Companies

Hint

An underground railway system is keeping track of customer travel times between different stations. They are using this data to calculate the average time it takes to travel from one station to another.

Implement the `UndergroundSystem` class:

- `void checkIn(int id, string stationName, int t)`
 - A customer with a card ID equal to `id`, checks in at the station `stationName` at time `t`.
 - A customer can only be checked into one place at a time.
- `void checkOut(int id, string stationName, int t)`
 - A customer with a card ID equal to `id`, checks out from the station `stationName` at time `t`.
- `double getAverageTime(string startStation, string endStation)`
 - Returns the average time it takes to travel from `startStation` to `endStation`.
 - The average time is computed from all the previous traveling times from `startStation` to `endStation` that happened **directly**, meaning a check in at `startStation` followed by a check out from `endStation`.
 - The time it takes to travel from `startStation` to `endStation` **may be different** from the time it takes to travel from `endStation` to `startStation`.
 - There will be at least one customer that has traveled from `startStation` to `endStation` before `getAverageTime` is called.

You may assume all calls to the `checkIn` and `checkOut` methods are consistent. If a customer checks in at time `t1` then checks out at time `t2`, then `t1 < t2`. All events happen in chronological order.

Example 1:

```
Input
["UndergroundSystem","checkIn","checkIn","checkIn","checkOut","checkOut","getAverageTime","getAverageTime","checkIn","getAverageTime","checkOut","getAverageTime"]
[[],[45,"Leyton",3],[32,"Paradise",0],[27,"Leyton",10],[45,"Waterloo",15],[27,"Waterloo",20],[32,"Cambridge",22],[32,"Paradise","Cambridge"],["Leyton","Waterloo"],[10,"Leyton",24],["Leyton","Waterloo"],[10,"Waterloo",38],["Leyton","Waterloo"]]
```

Code

Java

Auto

```
1 import java.util.*;
2
3 class UndergroundSystem {
4
5     // id -> check-in station and time
6     HashMap<Integer, CheckIn> checkIns = new HashMap<>();
7
8     // "start-end" -> [total time, number of trips]
9     HashMap<String, double[]> trips = new HashMap<>();
10
11     class CheckIn {
12         String station;
13         int time;
14
15         CheckIn(String station, int time) {
16             this.station = station;
17             this.time = time;
18         }
19     }
20
21     public UndergroundSystem() {}
22
23
24     public void checkIn(int id, String stationName, int t) {
25         checkIns.put(id, new CheckIn(stationName, t));
26     }
27
28     public void checkOut(int id, String stationName, int t) {
29         CheckIn in = checkIns.get(id);
30
31         String route = in.station + "-" + stationName;
32         double travellTime = t - in.time;
33
34         if (!trips.containsKey(route)) {
35             trips.put(route, new double[2]);
36         }
37         double[] trip = trips.get(route);
38         trip[0] += travellTime;
39         trip[1]++;
40     }
41
42     public double getAverageTime(String startStation, String endStation) {
43         String route = startStation + "-" + endStation;
44         double[] trip = trips.get(route);
45         return trip[0] / trip[1];
46     }
47 }
```

Testcase

Test Result

Accepted

Runtime: 21 ms

Case 1

Case 2

Problem List

1472. Design Browser History

Medium

Topics

Companies

Hint

You have a **browser** of one tab where you start on the `homepage` and you can visit another `url`, get back in the history number of `steps` or move forward in the history number of `steps`.

Implement the `BrowserHistory` class:

- `BrowserHistory(string homepage)` Initializes the object with the `homepage` of the browser.
- `void visit(string url)` Visits `url` from the current page. It clears up all the forward history.
- `string back(int steps)` Move `steps` back in history. If you can only return `x` steps in the history and `steps > x`, you will return only `x` steps. Return the current `url` after moving back in history **at most** steps.
- `string forward(int steps)` Move `steps` forward in history. If you can only forward `x` steps in the history and `steps > x`, you will forward only `x` steps. Return the current `url` after forwarding in history **at most** steps.

Example:

Input:
["BrowserHistory","visit","visit","visit","back","back","forward","visit","forward","back","back"]
[["leetcode.com"],["google.com"],["facebook.com"],["youtube.com"],[1],[1],[1],["linkedin.com"],[2],[2],[7]]

Output:
[null,null,null,null,"facebook.com","google.com","facebook.com",null,"linkedin.com","google.com","leetcode.com"]

Explanation:

```
BrowserHistory browserHistory = new BrowserHistory("leetcode.com");
browserHistory.visit("google.com"); // You are in "leetcode.com". Visit "google.com"
browserHistory.visit("facebook.com"); // You are in "google.com". Visit "facebook.com"
browserHistory.visit("youtube.com"); // You are in "facebook.com". Visit "youtube.com"
browserHistory.back(1); // You are in "youtube.com", move back to "facebook.com"
return "facebook.com"
browserHistory.back(1); // You are in "facebook.com", move back to "google.com" return
"google.com"
browserHistory.forward(1); // You are in "google.com", move forward to "facebook.com"
```

Code

```
1  import java.util.*;
2
3  class BrowserHistory {
4
5      ArrayList<String> history = new ArrayList<>();
6      int current = 0;
7
8      public BrowserHistory(String homepage) {
9          history.add(homepage);
10     }
11
12     public void visit(String url) {
13         // Remove forward history
14         while (history.size() > current + 1) {
15             history.remove(history.size() - 1);
16         }
17
18         history.add(url);
19         current++;
20     }
21
22     public String back(int steps) {
23         current = Math.max(0, current - steps);
24         return history.get(current);
25     }
26
27     public String forward(int steps) {
28         current = Math.min(history.size() - 1, current + steps);
29         return history.get(current);
30     }
31 }
```

Saved

Testcase

Test Result

Accepted

Runtime: 2 ms

Case 1

Problem List

705. Design HashSet

Description

Editorial

Solutions

Submissions

Easy

Topics

Companies

Design a HashSet without using any built-in hash table libraries.

Implement `MyHashSet` class:

- `void add(key)` Inserts the value `key` into the HashSet.
- `bool contains(key)` Returns whether the value `key` exists in the HashSet or not.
- `void remove(key)` Removes the value `key` in the HashSet. If `key` does not exist in the HashSet, do nothing.

Example 1:

Input

["MyHashSet", "add", "add", "contains", "contains", "add", "contains", "remove", "contains"]

[[], [1], [2], [1], [3], [2], [2], [2], [2]]

Output

[null, null, null, true, false, null, true, null, false]

Explanation

MyHashSet myHashSet = new MyHashSet();
myHashSet.add(1); // set = [1]
myHashSet.add(2); // set = [1, 2]
myHashSet.contains(1); // return True
myHashSet.contains(3); // return false, (not found)
myHashSet.add(2); // set = [1, 2]
myHashSet.contains(2); // return True
myHashSet.remove(2); // set = [1]
myHashSet.contains(2); // return false, (already removed)

Constraints:

- $0 \leq \text{key} \leq 10^6$
- At most 10⁴ calls will be made to `add`, `remove`, and `contains`.

4.1K

90

Code

Java

Auto

```
1 class MyHashSet {  
2  
3     boolean[] set = new boolean[1000001];  
4  
5     public MyHashSet() {  
6     }  
7  
8     public void add(int key) {  
9         set[key] = true;  
10    }  
11  
12    public void remove(int key) {  
13        set[key] = false;  
14    }  
15  
16    public boolean contains(int key) {  
17        return set[key];  
18    }  
19 }
```

Saved

Un 19, Col 2

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Problem

Submissions

Leaderboard

Discussions

Editorial

HackerLand University has the following grading policy:

- Every student receives a *grade* in the inclusive range from 0 to 100.
- Any *grade* less than 40 is a failing grade.

Sam is a professor at the university and likes to round each student's *grade* according to these rules:

- If the difference between the *grade* and the next multiple of 5 is less than 3, round *grade* up to the next multiple of 5.
- If the value of *grade* is less than 38, no rounding occurs as the result will still be a failing grade.

Examples

- grade* = 84 round to 85 (85 - 84 is less than 3)
- grade* = 29 do not round (result is less than 38)
- grade* = 57 do not round (60 - 57 is 3 or higher)

Given the initial value of *grade* for each of Sam's *n* students, write code to automate the rounding process.

Function Description

Complete the function *gradingStudents* with the following parameter(s):

- int grades[n]*: the grades before rounding

Returns

- int[n]*: the grades after rounding

Input Format

The first line contains a single integer, *n*, the number of students.

Each line *i* of the *n* subsequent lines contains a single integer, *grades[i]*.

Constraints

- $1 \leq n \leq 60$
- $0 \leq grades[i] \leq 100$

Sample Input 0

```
4
73
67
38
xx
```

```
15 // The function is expected to return an integer array.
16 // The function accepts INTEGER_ARRAY grades as parameter.
17 */
18
19 public static List<Integer> gradingStudents(List<Integer> grades) {
20
21     List<Integer> result = new ArrayList<>();
22
23     for (int grade : grades) {
24
25         if (grade < 38) {
26             result.add(grade);
27         } else {
28             int next = ((grade / 5) + 1) * 5;
29
30             if (next - grade < 3) {
31                 result.add(next);
32             } else {
33                 result.add(grade);
34             }
35         }
36     }
37
38     return result;
39 }
40
41
42 public class Solution {
43     public static void main(String[] args) throws IOException {
44         BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
45         BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));
46     }
47 }
```

Line: 39 Col: 4

Upload Code as File

☐ Test against custom input

RunCode

SubmitCode

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Using inheritance, one class can acquire the properties of others. Consider the following Animal class:

```
class Animal{
    void walk(){
        System.out.println("I am walking");
    }
}
```

This class has only one method, walk. Next, we want to create a Bird class that also has a fly method. We do this using extends keyword:

```
class Bird extends Animal {
    void fly() {
        System.out.println("I am flying");
    }
}
```

Finally, we can create a Bird object that can both fly and walk.

```
public class Solution{
    public static void main(String[] args){

        Bird bird = new Bird();
        bird.walk();
        bird.fly();
    }
}
```

The above code will print:

```
I am walking
I am flying
```

This means that a Bird object has all the properties that an Animal object has, as well as some additional unique properties.

The code above is provided for you in your editor. You must add a sing method to the Bird class, then modify the main method accordingly so that the code prints the following lines:

```
8      void walk(){
9          System.out.println("I am walking");
10     }
11 }
12
13 class Bird extends Animal{
14     void fly(){
15         System.out.println("I am flying");
16     }
17
18     void sing() {
19         System.out.println("I am singing");
20     }
21 }
22
23 public class Solution{
24
25     public static void main(String args[]){
26
27         Bird bird = new Bird();
28         bird.walk();
29         bird.fly();
30         bird.sing();
31     }
32 }
33 }
```

Line: 17 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

Write the following code in your editor below:

1. A class named `Arithmetic` with a method named `add` that takes **2** integers as parameters and returns an integer denoting their sum.
2. A class named `Adder` that inherits from a superclass named `Arithmetic`.

Your classes should not be `public`.

Input Format

You are not responsible for reading any input from `stdin`; a locked code stub will test your submission by calling the `add` method on an `Adder` object and passing it **2** integer parameters.

Output Format

You are not responsible for printing anything to `stdout`. Your `add` method must return the sum of its parameters.

Sample Output

The main method in the `Solution` class above should print the following:

```
My superclass is: Arithmetic
42 13 20
```

```

3  import java.text.*;
4  import java.math.*;
5  import java.util.regex.*;
6  class Arithmetic {
7      int add(int a, int b) {
8          return a + b;
9      }
10 }
11
12 class Adder extends Arithmetic {
13 }
14 public class Solution {
15     public static void main(String []args){
16         // Create a new Adder object
17         Adder a = new Adder();
18
19         // Print the name of the superclass on a new line
20         System.out.println("My superclass is: " + a.getClass().getSuperclass().getName());
21
22         // Print the result of 3 calls to Adder's 'add(int,int)' method as 3 space-separated integers:
23         System.out.print(a.add(10,32) + " " + a.add(10,3) + " " + a.add(10,10) + "\n");
24     }
25 }
26
```

Line: 14 Col: 23

☐ Test against custom input

Congratulations!



Search



ENG IN 09:44 02-09-2026

Generic methods are a very efficient way to handle multiple datatypes using a single method. This problem will test your knowledge on Java Generic methods.

Let's say you have an integer array and a string array. You have to write a **single** method `printArray` that can print all the elements of both arrays. The method should be able to accept both integer arrays or string arrays.

You are given code in the editor. Complete the code so that it prints the following lines:

```
1
2
3
Hello
World
```

Do not use method overloading because your answer will not be accepted.

```
5 class Printer {
6
7     public <T> void printArray(T[] array) {
8         for (T element : array) {
9             System.out.println(element);
10        }
11    }
12 }
13 public class Solution {
14
15
16     public static void main( String args[] ) {
17         Printer myPrinter = new Printer();
18         Integer[] intArray = { 1, 2, 3 };
19         String[] stringArray = { "Hello", "World" };
20         myPrinter.printArray(intArray);
21         myPrinter.printArray(stringArray);
22         int count = 0;
23
24         for (Method method : Printer.class.getDeclaredMethods()) {
25             String name = method.getName();
26
27             if(name.equals("printArray"))
28                 count++;
29         }
30
31         if(count > 1) System.out.println("Method overloading is not allowed!");
32     }
33 }
34 }
```

Lines: 22 Cols: 2

☐ Test against custom input

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Problem List

Submit

0

Premium

Description

Editorial

Solutions

Submissions

1600. Throne Inheritance

MediumTopicsCompaniesHint

A kingdom consists of a king, his children, his grandchildren, and so on. Every once in a while, someone in the family dies or a child is born.

The kingdom has a well-defined order of inheritance that consists of the king as the first member. Let's define the recursive function `Successor(x, curOrder)`, which given a person `x` and the inheritance order so far, returns who should be the next person after `x` in the order of inheritance.

```
Successor(x, curOrder):
    if x has no children or all of x's children are in curOrder:
        if x is the king return null
        else return Successor(x's parent, curOrder)
    else return x's oldest child who's not in curOrder
```

For example, assume we have a kingdom that consists of the king, his children Alice and Bob (Alice is older than Bob), and finally Alice's son Jack.

1. In the beginning, `curOrder` will be `["king"]`.
2. Calling `Successor(king, curOrder)` will return Alice, so we append to `curOrder` to get `["king", "Alice"]`.
3. Calling `Successor(Alice, curOrder)` will return Jack, so we append to `curOrder` to get `["king", "Alice", "Jack"]`.
4. Calling `Successor(Jack, curOrder)` will return Bob, so we append to `curOrder` to get `["king", "Alice", "Jack", "Bob"]`.
5. Calling `Successor(Bob, curOrder)` will return null. Thus the order of inheritance will be `["king", "Alice", "Jack", "Bob"]`.

Using the above function, we can always obtain a unique order of inheritance.

Implement the `ThroneInheritance` class:

- `ThroneInheritance(string kingName)` Initializes an object of the `ThroneInheritance` class. The name of the king is given as part of the constructor.
- `void birth(string parentName, string childName)` Indicates that `parentName` gave birth to `childName`.
- `void death(string name)` Indicates the death of `name`. The death of the person doesn't affect the `Successor` function nor the current inheritance order. You can treat it as just marking the person as dead.
- `string[] getInheritanceOrder()` Returns a list representing the current order of inheritance **excluding** dead people.

Coder

Auto

```
1 import java.util.*;
2
3 class ThroneInheritance {
4
5     private String king;
6     private Map<String, List<String>> children;
7     private Set<String> dead;
8
9     public ThroneInheritance(String kingName) {
10         king = kingName;
11         children = new HashMap<>();
12         dead = new HashSet<>();
13     }
14
15     public void birth(String parentName, String childName) {
16         children.putIfAbsent(parentName, new ArrayList<>());
17         children.get(parentName).add(childName);
18     }
19
20     public void death(String name) {
21         dead.add(name);
22     }
23
24     public List<String> getInheritanceOrder() {
25         List<String> order = new ArrayList<>();
26         dfs(king, order);
27         return order;
28     }
29
30     private void dfs(String person, List<String> order) {
31
32         if (!dead.contains(person)) {
33             order.add(person);
34         }
35     }
36 }
```

SavedLn 42, Col 2

Testcase

Test Result

AcceptedRuntime: 6 ms

Case 1

322

19